

SQL Performance Tuning – Notes

1. Introduction

SQL performance tuning refers to the process of optimizing database queries and structures to improve the speed and efficiency of data retrieval and manipulation. When queries are not optimized, they can consume excessive system resources such as CPU, memory, and disk I/O. This leads to slow database performance, longer response times, and sometimes system downtime.

Effective SQL performance tuning ensures that queries execute quickly, server load is minimized, and database resources are used efficiently. It plays a crucial role in maintaining stable and scalable database systems, especially in environments with large datasets and multiple concurrent users.

2. Factors Affecting SQL Query Performance 1. Table Size

Large tables containing millions of records can slow down query execution. Queries that scan large portions of a table require more processing time and system resources.

2. Joins

Joining multiple tables increases the complexity of query execution. Complex joins may cause large intermediate result sets, increasing processing time.

3. Aggregations

Operations such as SUM, AVG, COUNT, and GROUP BY require additional computation, especially when applied to large datasets.

4. Concurrency

When many users run queries simultaneously, the database server must handle multiple requests at once. This can lead to contention for resources and slower query execution.

5. Indexes

Indexes improve data retrieval speed by allowing the database to locate rows quickly. However, poorly designed indexes can reduce performance, especially during insert or update operations.

3. Methods to Identify Slow SQL Queries

3.1 Execution Plan

An execution plan shows how the database engine executes a query. It provides information about operations such as table scans, index usage, and join methods.

Steps to generate an execution plan in SQL Server Management Studio:

1. Open a new Database Engine Query window.
2. Write the SQL query.
3. Enable the "Include Actual Execution Plan" option.
4. Execute the query.
5. Review the execution plan displayed in the results pane.

Execution plans help identify inefficient operations such as full table scans or missing indexes.

3.2 Monitoring Resource Usage

Database performance depends heavily on system resources including CPU, memory, and disk activity. Tools like Windows Performance Monitor allow administrators to observe these metrics and detect performance bottlenecks. Monitoring both SQL Server counters and operating system metrics helps identify correlations between resource usage and query performance issues.

3.3 Dynamic Management Views (DMVs)

SQL Server provides Dynamic Management Views that allow administrators to analyze query execution statistics and identify slow-running queries.

Example DMV: sys.dm_exec_query_stats

These views provide information such as:

- Execution count
- Query execution time
- Resource consumption
- Query plans

Using DMVs helps identify inefficient queries that need optimization.

4. SQL Query Optimization Techniques

4.1 Avoid Using SELECT *

Using SELECT * retrieves all columns from a table, even when only a few are required. This increases data transfer and processing time.

Inefficient:

```
SELECT * FROM GeeksTable;
```

Efficient:

```
SELECT FirstName, LastName, Address, City, State, Zip  
FROM GeeksTable;
```

4.2 Avoid SELECT DISTINCT When Possible

SELECT DISTINCT removes duplicate records but requires additional processing. Instead, design queries so that duplicates are naturally avoided.

Inefficient:

```
SELECT DISTINCT FirstName, LastName, State  
FROM GeeksTable;
```

Efficient:

```
SELECT FirstName, LastName, State  
FROM GeeksTable  
WHERE State IS NOT NULL;
```

4.3 Use INNER JOIN Instead of WHERE for Joins

Using explicit join syntax improves readability and often leads to better query optimization by the database engine.

Inefficient:

```
SELECT GFG1.CustomerID, GFG1.Name, GFG1.LastSaleDate  
FROM GFG1, GFG2  
WHERE GFG1.CustomerID = GFG2.CustomerID;
```

Efficient:

```
SELECT GFG1.CustomerID, GFG1.Name, GFG1.LastSaleDate  
FROM GFG1  
INNER JOIN GFG2  
ON GFG1.CustomerID = GFG2.CustomerID;
```

4.4 Use WHERE Instead of HAVING

The WHERE clause filters records before grouping, while HAVING filters results after aggregation. Filtering earlier reduces the amount of data processed.

Inefficient:

```
SELECT CustomerID, Name  
FROM GFG1  
GROUP BY CustomerID, Name  
HAVING LastSaleDate BETWEEN '2019-01-01' AND '2019-12-31';
```

Efficient:

```
SELECT CustomerID, Name  
FROM GFG1  
WHERE LastSaleDate BETWEEN '2019-01-01' AND '2019-12-31'  
GROUP BY CustomerID, Name;
```

4.5 Limit Wildcards in Search Conditions

Using % at the beginning of a pattern prevents the use of indexes and slows down searches.

Inefficient:

```
SELECT City FROM GeekTable
```

```
WHERE City LIKE '%No%';
```

Efficient:

```
SELECT City FROM GeekTable
```

```
WHERE City LIKE 'No%';
```

4.6 Use LIMIT for Testing Queries

The LIMIT clause restricts the number of returned rows. It is useful when testing queries on large tables to avoid unnecessary processing.

Example:

```
SELECT CustomerID, Name
```

```
FROM Customers
```

```
LIMIT 10;
```

4.7 Run Heavy Queries During Off-Peak Hours

Running large analytical queries during periods of low system usage reduces the impact on other users. This is often scheduled overnight or during maintenance windows.

5. Index Tuning

Index tuning involves selecting appropriate columns to index so that query execution becomes faster.

Important considerations when creating indexes:

- **Short Indexes:** Smaller indexes reduce disk usage and allow faster comparisons.
- **High Selectivity:** Columns with many unique values make better indexes.
- **Clustered Indexes:** Store table data in sorted order based on the indexed column.
- **Static Columns:** Columns that rarely change are good candidates for clustered indexes.

While indexes improve read performance, they may slow down insert and update operations because indexes must also be updated.

6. SQL Performance Tuning Tools

Several tools help monitor database performance and identify slow queries.

SQL Sentry (SolarWinds)

Provides real-time monitoring, performance analysis, and query optimization insights.

SQL Profiler (Microsoft)

Captures and analyzes SQL Server events, helping identify slow queries and performance issues.

SQL Index Manager (Red Gate)

Assists in analyzing and optimizing database indexes.

SQL Diagnostic Manager (IDERA)

Provides comprehensive monitoring of SQL Server performance and resource usage.

These tools help database administrators detect inefficiencies and improve query performance.

Database Normalization – Notes 1. Introduction

Database normalization is a systematic process used to organize data in a database to reduce redundancy and improve data integrity. The main objective is to divide large tables into smaller related tables while maintaining logical relationships between them.

Normalization ensures that data is stored efficiently and prevents anomalies that can occur during data insertion, deletion, and updates.

2. Problems Without Normalization

Insertion Anomaly

Occurs when new data cannot be inserted because other required data is missing.

Example:

If a department table requires employee details, a department without employees cannot be stored.

Update Anomaly

Occurs when the same data appears in multiple rows and must be updated in each location.

Example:

If a department location changes, all rows containing that department must be updated.

Deletion Anomaly

Occurs when deleting one record unintentionally removes additional information.

Example:

If the last employee in a department is deleted, the department information may also be lost.

Data Redundancy

The same data is stored multiple times, increasing storage requirements and creating consistency problems.

3. Need for Normalization

Normalization helps to:

- Reduce duplicate data
- Improve data consistency
- Simplify database maintenance
- Prevent insertion, update, and deletion anomalies

- Improve database structure and flexibility

4. Features of Database Normalization

Elimination of Data Redundancy

Normalization removes repeated data and ensures each piece of information is stored only once.

Data Consistency

Ensures that all references to the same data remain consistent across the database.

Simplified Data Management

Breaking large tables into smaller ones makes data easier to manage and maintain.

Improved Database Design

A normalized structure improves scalability and flexibility.

Avoidance of Update Anomalies

Changes to data only need to be made in one place, preventing inconsistencies.

Standardized Data Organization

Data is structured logically with clear relationships between tables.

5. Normal Forms

Normalization is achieved through different levels called normal forms.

First Normal Form (1NF)

A table is in 1NF if:

- Each column contains atomic values.
- There are no repeating groups or multi-valued attributes.

Second Normal Form (2NF)

A table is in 2NF if:

- It is already in 1NF.
- Every non-key attribute is fully dependent on the primary key.

This eliminates partial dependency.

Third Normal Form (3NF)

A table is in 3NF if:

- It is already in 2NF.
- There are no transitive dependencies between non-key attributes.

This ensures non-key attributes depend only on the primary key.

Boyce-Codd Normal Form (BCNF)

BCNF is a stronger version of 3NF.

Conditions:

- The table must already be in 3NF.
- For every functional dependency $X \rightarrow Y$, X must be a super key.

Fourth Normal Form (4NF)

A table is in 4NF if:

- It satisfies BCNF.
- It has no multi-valued dependencies.

Fifth Normal Form (5NF) A

table is in 5NF if:

- It is already in 4NF.
- It cannot be further decomposed without losing information.

This form deals with complex join dependencies.

Conclusion

SQL performance tuning and database normalization are fundamental techniques for building efficient database systems. Performance tuning focuses on improving query execution and resource utilization, while normalization focuses on organizing data structures to eliminate redundancy and maintain consistency. Together, they ensure reliable, scalable, and high-performing database environments.

Task: find KPIs on the iris dataset and penguin dataset using SQL

Iris dataset kpi:

The screenshot shows the MySQL Workbench interface. The 'Schemas' pane on the left lists databases like 'companydb', 'customerdb', and 'dataset'. The 'dataset' database is selected, showing tables 'iris', 'penguins', and 'species'. The 'Query' tab is active, displaying the following SQL query:

```
1 -- Iris dataset KPIs
2 use dataset;
3
4 select * from iris dataset;
5
6 -- Total Records
7 select count(*) as total_records;
```

The 'Results' pane shows the output of the query, which is a single row with the value 150 for 'total_records'.

total_records
150

The 'Action Output' pane at the bottom shows the execution details: '5 17:52:52 SELECT COUNT(*) AS total_records FROM iris LIMIT 5, 1000'. The status is '1 row(s) returned' and the duration is '0:00:00 / 0:00:00'.

The screenshot shows the MySQL Workbench interface with a more complex SQL query for KPIs on the iris dataset. The 'Query' tab displays the following SQL query:

```
107 -- Iris dataset KPIs
108 use dataset;
109
110 -- Total Records
111 select count(*) as total_records,
112 species,
113 count(*) as species_count,
114 round(count(*)/total_records, 2) as species_percentage,
115 round(avg(sepal_length), 2) as avg_sepal_length,
116 round(avg(sepal_width), 2) as avg_sepal_width,
117 round(avg(petal_length), 2) as avg_petal_length,
118 round(avg(petal_width), 2) as avg_petal_width
119 from iris;
120 group by species;
```

The 'Results' pane shows the output of the query, which is a table with 8 columns: 'total_records', 'species', 'species_count', 'species_percentage', 'avg_sepal_length', 'avg_sepal_width', 'avg_petal_length', and 'avg_petal_width'. The table has 3 rows, one for each species: 'setosa', 'versicolour', and 'virginica'.

total_records	species	species_count	species_percentage	avg_sepal_length	avg_sepal_width	avg_petal_length	avg_petal_width
50	setosa	50	0.33	5.01	3.42	1.46	0.24
50	versicolour	50	0.33	5.94	4.53	4.35	1.31
50	virginica	50	0.33	6.76	3.76	4.26	1.53

The 'Action Output' pane at the bottom shows the execution details: '5 17:54:52 SELECT COUNT(*) AS total_records FROM iris LIMIT 5, 1000'. The status is '1 row(s) returned' and the duration is '0:00:00 / 0:00:00'.

Penguin dataset kpi:

The screenshot shows the MySQL Workbench interface. The left sidebar displays the database schema with the 'penguins' table selected under the 'penguins' database. The main query editor contains the following SQL statement:

```

-- kpi for penguin dataset using kpi
1
2
3
4 select * from penguins;
5

```

The 'Result Grid' pane shows the first 20 rows of the penguins table. The columns are: species, island, id, body_mass_g, flipper_length_mm, wing_length_mm, and sex. The data includes various species like Adelia, Chinstrap, and Gentoo, with their respective measurements and sex (Male or Female).

The 'Status Bar' at the bottom indicates that 8 rows were selected from the penguins table, and the query was executed successfully.

The screenshot shows the MySQL Workbench interface with a more complex query. The query editor contains the following SQL statement:

```

-- KPI for penguin dataset using kpi
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

The 'Result Grid' pane shows the first 20 rows of the calculated KPIs. The columns are: total_penguins, avg_body_mass_g, avg_flipper_length_mm, avg_wing_length_mm, male_percentage, and female_percentage. The data shows that there are 344 penguins in total, with an average body mass of 4177.32g, average flipper length of 188.71mm, average wing length of 41.57mm, 48.94% male, and 47.97% female.

The 'Status Bar' at the bottom indicates that 7 rows were selected from the penguins table, and the query was executed successfully.